Guidance of the Circuit Model Set Up

This document is a guide to building a level-1 s-function to simulate a basic circuit. The manuscript takes the buck converter as an example. All the example codes are based on the buck converter S-function construction. The full version of the codes is displayed at the end of the manuscript. Not all the functions are needed to be filled. Specifically, only "**mdlInitializeSiezes**", "**mdlUpdate**", and "**mdlOutputs**" are required to modify. They are responsible for the **initializations**, **variable update** and **control of output signals**, respectively.

Function Title Explanation:

function [sys,x0,str,ts,simStateCompliance] = sfuntmlp(t,x,u,flag)

The code above is the first line of the code. The only part that needs to be modified is the "**sfuntmlp**", which is the name of the function. The function name should be exactly the same as the file name. For example, in the case of the buck converter, the first line of the code is:

function [sys,x0,str,ts,simStateCompliance] = buck_sfunct(t,x,u,flag)

Where the "**buck_sfunct**" is the name of the s-function and also the file name.

Initialization of the S-function:

To simulate the system, the initialization of the system is critical. There are several parameters need to be determined in the "**mdlInitializeSizes**" function.

1. **sizes.NumContStates**
   This parameter sets the number of continuous variables in the system. Since all the simulations are discrete, the number of the continuous variable is 0.
   i.e. sizes.NumContStates = 0;

2. **sizes.NumDiscStates**
   This parameter sets the number of the discrete variables. Normally, the discrete variables are also the variables in the output array "**x**". The value of this parameter depends on the number of discrete variables which we want to monitor. Normally, these variables are voltages and the currents of the capacitors and inductors, respectively. For example, in a Buck converter, we want to monitor the **inductor currents (x[1])** and the **output capacitor voltage (x[2])**, thus, the number of discrete variables would be 2:
   sizes.NumDiscStates = 2;

3. **sizes.NumOutputs**
   This parameter sets the number of the output of the built s-function. The value should not be larger than the value of the "**sizes.NumDiscStates**". For example, in the Buck converter case, if we want to plot the inductor current and the capacitor voltage, the value should be set to 2:
   sizes.NumOutputs = 2;

4. **sizes.NumInputs**
   This parameter sets the number of the input of the s-function. For example, in the Buck converter case, the only input is the drive signal of the switch, thus the value should be 1:
   sizes.NumInputs = 1;

5. **sizes.DirFeedthrough**
   This value determines whether the output of the system (x) depends on the input. If the output depends on the input, the value should be 1, otherwise, it is 0. For example, in the Buck converter, the inductor current depends on the input signal, thus the value should be 1:

   sizes.DirFeedthrough = 1;

6. **sizes.NumSampleTimes**
   This value determines the number of the sample time. Normally, it is 1. It could also be 0. However, if the parameter is 1, there are 3 more values should also be determined:
   a) **"x0"**: It is the initial value of the discrete values (**sizes.NumDiscStates**) in the vector form (both column vector and row vector are ok). For example, in the Buck converter case, the values of the inductor current and the capacitor voltage are all 0. **i.e. x0 = [0; 0];**
   b) **"str"**: Normally, leave it as blank. **i.e. str = [ ];**
   c) **"ts"**: It is the sample time, which has the form of "**ts = [period, time]**". The "period" is the sampling cycle, and the "time" is the starting time. For each sampling time can be calculated as: "**time + n*period**". For example, in the Buck converter case, the sampling cycle is 0.000001s and it starts from 0s of the simulating. **i.e. ts = [0.000001 0];**

Since we are using the discrete simulation, the function of "**mdlDerivatives**" would be empty.

Contents in the update function (mdlUpdate)

The following parts are the guidance for modelling each electrical component. Since the discrete simulation is adapted, **all the equations are in the function of "mdlUpdate"**, which updates the discrete values of the s-function.

Before writing the equations for each component, several local constant variables should be claimed in the **mdlUpdate** function, including **Va**, **L**, **C**, **Ts** and **R**. **Va** is the input voltage; **L** is the inductor value; **C** is the capacitor value; **Ts** is the sample time of the simulation (normally 1/20 of the switching period); **R** is the load used in the simulation, which can be calculated by:

$R = \frac{Vout^2}{P}$, where the **P** the power of the system and the **Vout** is the output voltage.

Voltage Source:

The **input** of the voltage source is **current** and the **output** is **voltage**. For an independent voltage source, the output of the voltage source is a constant which is independent of the input current. For example, in the case of the buck converter, the code could be:

Vs = Va;

Where **Vs** is the output voltage; **Va** is the constant input voltage defined before.

Switch Network:

The switch here means the switches network, which may contain multiple switches. The **inputs** of the switch network are the **voltage** and **current**. The **outputs** of the switch network are also the **voltage** and **current**. Since the switches are controlled by some control signals ("s"), the outputs of the switches network are also controlled by such control signals ("s"). Normally, the control signal has two values: "1" and "0". When the switch is on, the value of "s" is 1, while the "s" is 0 when the switch is off.

$$s = \begin{cases} 1 & \text{switch on} \\ 0 & \text{switch off} \end{cases}$$

For example, in the case of the buck converter, the input voltage of the network is the voltage source, and the input current is the inductor current. When the switch is on, the voltage output of the switch network is the voltage source, and the output is 0 when the switch is off. Similarly, the current output is the inductor current when the switch is on, and the output current is 0 when the switch is off. Thus, the codes can be:

Vswout = u*Va;

Iswout = u*x(1);

Where **Vswout** is the **switch output voltage**; **Iswout** is the **switch output current**; **u** is the input of the s-function; which is the control signal of the switches; **x(1)** is the **inductor current**, which uses the **discrete value**.

Load:

The **input** of the load is **voltage** and the **output** is **current**. The output current equals the voltage of the load divided by the value of the load (resistor). Normally, the voltage is one of the discrete values. For example, in the case of the buck converter, the voltage drops across the resistor is capacitor voltage, which is the second element in the "x" array (x[2]). Thus, the code could be:

I_R = x(2)/R;

Where the **I_R** is the **load output current**; **R** is the **load value** calculated before; **x(2)** is the **capacitor voltage**, which uses the **discrete value**.

Inductor:

The input of the inductor is voltage, and the output is current. The relationship between the output current and the input voltage is:

$$V_L = L\frac{dI_L}{dt}$$

Where the I_L is the inductor current, V_L is the inductor voltage, and L is the inductor value. Rearrange the equations (assumes zero initial conditions):

$$I_L = \frac{1}{L}\int V_L dt$$

However, the equation above is in the continuous domain. Converting the equation into the discrete domain and rearranging the equation:

$$I_L(n+1) = I_L(n) + dI_L = I_L(n) + \frac{V_L(n)}{L}T_s$$

Where **Ts** is the **sample time** for the simulation, $I_L(n+1)$ is the updated inductor current, and $I_L(n)$ is the discrete variable, i.e. x(1) in the buck converter example.

For example, in the case of the buck converter, the inductor voltage equals the output voltage minus the capacitor voltage:

$$V_L = V_{swout} - V_C$$

Where the V_C could be represented using x(2).
The code could be:

I_L = x(1) + (Vswout-x(2))/L*Ts;

Where the **I_L** is the **inductor output current**; **Vswout** is the **switch output voltage; L** is the **inductor value**; **Ts** is the **sample time**; **x(1)**, **x(2)** are the **inductor current** and **capacitor voltage** respectively, which use the **discrete values**.

Capacitor:

The input of the capacitor is current, and the output is voltage. The relationship between the output voltage and the input current is:

$$I_C = C\frac{dV_C}{dt}$$

Where the I_L is the inductor current, V_L is the inductor voltage, and L is the inductor value. Rearrange the equations (assumes zero initial conditions):

$$V_C = \frac{1}{C}\int I_C dt$$

However, the equation above is in the continuous domain. Converting the equation into the discrete domain and rearranging the equation:

$$V_C(n+1) = V_C(n) + dV_C = V_C(n) + \frac{I_C(n)}{C}T_s$$

Where Ts is the sample time for the simulation, $I_L(n+1)$ is the updated inductor current, and $I_L(n)$ is the discrete variable, i.e. x(1) in the buck converter example.
For example, in the case of the buck converter, the capacitor current equals the inductor current minus the load current:

$$I_C = I_L - I_R$$

Where the I_C is the capacitor current, I_L is the inductor current and the I_R is the load current.
Where the I_L could be represented using x(1).
The code could be:

V_C = x(2) + (x(1)-I_R)/C*Ts;

Where the **V_C** is the **capacitor output voltage**; **I_R** is the **load output current; C** is the **capacitor value**; **Ts** is the **sample time**; **x(1)**, **x(2)** are the **inductor current** and **capacitor voltage** respectively, which use the **discrete values**.

Output of the update function:

This section deals with the output of the update function. In theory, the output could be anything. For example, in the case of the buck converter, let the output be the inductor current and the capacitor current. The code could be:

sys = [I_L; V_C];

Output Function (mdlOutputs)

This function controls the output signal of the s-function. Normally they are the pre-defined discrete variables. For example, in the case of the buck converter, the output signals would be the inductor current and the capacitor voltage. The codes would be:

sys = [x(1); x(2)];

Note: the number of the output signals should be consistent with the parameter of "**sizes.NumOutputs**" which is predefined in the initialization part.

Examples of the codes for buck converter:

```
function [sys,x0,str,ts,simStateCompliance] = buck_sfunc(t,x,u,flag)

switch flag,
  case 0,
    [sys,x0,str,ts,simStateCompliance]=mdlInitializeSizes;

  case 1,buck;

  case 2,
    sys=mdlUpdate(t,x,u);

  case 3,
    sys=mdlOutputs(t,x,u);

  case 4,
    sys=mdlGetTimeOfNextVarHit(t,x,u);

  case 9,
    sys=mdlTerminate(t,x,u);

  otherwise
    DAStudio.error('Simulink:blocks:unhandledFlag', num2str(flag));

end

% mdlInitializeSizes
% Return the sizes, initial conditions, and sample times for the S-
function.
function [sys,x0,str,ts,simStateCompliance]=mdlInitializeSizes

sizes = simsizes;

sizes.NumContStates  = 0;   % # of continouse variables
```

```
sizes.NumDiscStates  = 2;   % number of discrete variables
sizes.NumOutputs     = 2;   % number of voltage
sizes.NumInputs      = 1;   % inputs: "s" signal
sizes.DirFeedthrough = 1;
sizes.NumSampleTimes = 1;   % at least one sample time is needed

sys = simsizes(sizes);

% initial conditions of the inductor current and capacitor voltage
x0 = [0; 0];

% str is always an empty matrix
str = [];

% initialize the array of sample times
Ts = 0.000001;
ts = [Ts 0];
% ts = [period offset]

simStateCompliance = 'UnknownSimState';

% end mdlInitializeSizes

function sys=mdlDerivatives(t,x,u)

sys = [];
% end mdlDerivatives

% mdlUpdate
% Handle discrete state updates, sample time hits, and major time step
% requirements.
function sys=mdlUpdate(t,x,u)
% constants
Ts = 0.000001;
Va = 15;
L = 8.375e-4;
C = 2.5e-4;
R = 2.5;
% characteristic functions
% Source
Vs = Va;
% Switches
Vswout = u*Va;
Iswout = u*x(1);
```

```
% Load
I_R = x(2)/R;
% Inductor & Capacitor
I_L = x(1) +  (Vswout - x(2))/L*Ts;
V_C = x(2) + (x(1) - I_R)/C*Ts;

sys = [I_L ; V_C];

% end mdlUpdate

% mdlOutputs
% Return the block outputs.
function sys=mdlOutputs(t,x,u)

sys = [x(1) ; x(2)];
% end mdlOutputs

% mdlGetTimeOfNextVarHit
% Return the time of the next hit for this block.  Note that the result is
% absolute time.  Note that this function is only used when you specify a
% variable discrete-time sample time [-2 0] in the sample time array in
% mdlInitializeSizes.

function sys=mdlGetTimeOfNextVarHit(t,x,u)
sampleTime = 1;    %  Example, set the next hit to be one second later.
sys = t + sampleTime;
% end mdlGetTimeOfNextVarHit

% mdlTerminate
% Perform any end of simulation tasks.

function sys=mdlTerminate(t,x,u)

sys = [];
% end mdlTerminate
```